

ΝΕΥΡΩΝΙΚΑ ΔΙΚΤΥΑ - 1η ΕΡΓΑΣΙΑ

Ζαχαριάδου Ευάγγελος - ΑΕΜ: 10649

Νοέμβριος 2024

1 Εισαγωγή

Στην εργασία αυτή υλοποιείται ένα Νευρωνικό Δίκτυο Εμπρόσθιας Τροφοδότησης (feedforward NN) MLP, το οποίο εκπαιδεύεται με τον αλγόριθμο back-propagation. Το μοντέλο αυτό εκπαιδεύεται για να επιλύει οποιοδήποτε πρόβλημα κατηγοριοποίησης (classification) με επιβλεπόμενη μάθηση.

1.1 Δίκτυο MLP - Multilayer Perceptron

Το MLP είναι ένα είδος νευρωνικού δικτύου που χρησιμοποιείται σε προβλήματα κατηγοριοποίησης όπως το δικό μας. Αποτελείται από το Input Layer, το οποίο δέχεται τα δεδομένα εισόδου, τα Hidden Layers, τα οποία επεξεργάζονται τα δεδομένα και μαθαίνουν αφηρημένα χαρακτηριστικά και το Output Layer, το οποίο παράγει την τελική πρόβλεψη. Το κύριο χαρακτηριστικό του δικτύου MLP είναι ότι είναι πλήρως διασυνδεδεμένο, δηλαδή η κάθε νευρώνας μιας στρώσης συνδέεται με όλους τους νευρώνες της επόμενης στρώσης μέσω βαρών.

1.1.1 Τρόπος Λειτουργίας του MLP.

Τα δεδομένα εισόδου περνούν από τις κρυφές στρώσεις, επεξεργάζονται από τις συναρτήσεις ενεργοποίησης, και φτάνουν στην έξοδο. Στην έξοδο υπολογίζεται το σφάλμα, δηλαδή η διαφορά μεταξύ της πρόβλεψης του δικτύου και της πραγματικής τιμής και έπειτα το σφάλμα διαδίδεται προς τα πίσω μέσω του δικτύου για την προσαρμογή των βαρών.

1.2 Δομή Δεδομένων CIFAR-10.

Για την εκπαίδευση και την κατηγοριοποίηση χρησιμοποιείται η βάση δεδομένων Cifar-10. Το Cifar-10 dataset αποτελείται από 60.000 εικόνες 32*32 pixels, οι οποίες ανήκουν σε 10 κλάσεις, δηλαδή κάθε κλάση έχει 6.000 εικόνες. Από τις 60.000 συνολικές φωτογραφίες, οι 50.000 χρησιμοποιούνται για την εκπαίδευση, και οι υπόλοιπες 10.000 εικόνες χρησιμοποιούνται για δοκιμές. Το CIFAR-10 χρησιμοποιείται συχνά για τη μέτρηση της απόδοσης και τη βελτιστοποίηση αλγορίθμων μηχανικής μάθησης σε προβλήματα ταξινόμησης εικόνων. Οι 50.000 εικόνες εκπαίδευσης χωρίζονται σε 5 training batches, όπου σε κάθε batch, οι εικόνες είναι διατεταγμένες σε τυχαία σειρά, δηλαδή κάποια training batches μπορεί να έχουν ελαφρώς περισσότερες εικόνες από κάποια κατηγορία, αλλά συνολικά σε όλα τα training batches υπάρχει ισορροπία με 5,000 εικόνες ανά κατηγορία. Έχουμε επίσης ένα test batch, στο οποίο περιέχονται οι υπόλοιπες 10.000 εικόνες που χρησιμοποιούνται για δοκιμές.

2 Ο Αλγόριθμος

Όπως προαναφέρθηκε, ο αλγόριθμος αφορά την υλοποίηση ενός Multilayer Perceptron (MLP) για κατηγοριοποίηση εικόνων από το dataset CIFAR-10.

2.1 Εισαγωγή Βιβλιοθηκών

Οι βιβλιοθήκες που χρησιμοποιήσα είναι οι εξής:

- **numpy** η οποία χρησιμοποιείται για αριθμητικούς υπολογισμούς.

- **pickle** η οποία χρησιμοποιείται για τη φόρτωση δεδομένων από αρχεία CIFAR-10.
- **os** η οποία χρησιμοποιείται για διαχείριση αρχείων και μονοπατιών.
- **time** η οποία υπολογίζει το χρόνο εκτέλεσης του προγράμματος.

2.2 Φόρτωση και προεπεξεργασία δεδομένων

Στην αρχή του προγράμματος ορίζεται η συνάρτηση `load_cifar10_data` η οποία φορτώνει το dataset CIFAR-10, το οποίο αποτελείται από τα εκπαιδευτικά δεδομένα (training data) που είναι 50.000 εικόνες και τα δεδομένα δοκιμής (test data) που είναι 10.000 εικόνες. Επιστρέφει τα δεδομένα εκπαίδευσης κανονικοποιημένα στο $[0,1]$ (`train_data`), τις ετικέτες εκπαίδευσης (`train_labels`), τα δεδομένα δοκιμής κανονικοποιημένα στο $[0,1]$ (`test_data`) και τις ετικέτες εκπαίδευσης (`test_labels`).

```
# φόρτωση των δεδομένων CIFAR-10
def load_cifar10_data(data_dir):
    def unpickle(file):
        with open(file, 'rb') as fo:
            dict = pickle.load(fo, encoding='bytes')
        return dict

    train_data = [] # αποθηκεύει τα δεδομένα εικόνες
    train_labels = [] # αποθηκεύει τις ετικέτες για κάθε εικόνα
    for i in range(1, 6):
        batch = unpickle(os.path.join(data_dir, f"data_batch_{i}"))
        train_data.append(batch[b'data']) # δεδομένα εικόνες
        train_labels += batch[b'labels'] # label εικόνες
    train_data = np.vstack(train_data).astype(np.float32) / 255.0 # κανονικοποίηση στο [0,1]
    train_labels = np.array(train_labels)

    test_batch = unpickle(os.path.join(data_dir, "test_batch"))
    test_data = test_batch[b'data'].astype(np.float32) / 255.0
    test_labels = np.array(test_batch[b'labels'])

    # επιστρέφονται τα δεδομένα εκπαίδευσης, οι ετικέτες εκπαίδευσης, τα δεδομένα δοκιμής
    # και οι ετικέτες δοκιμής.
    return train_data, train_labels, test_data, test_labels
```

Σχήμα 1: Συνάρτηση `load_cifar10_data`

2.3 Ορισμός συναρτήσεων ενεργοποίησης.

Στην συνέχεια έχουμε τον ορισμό των συναρτήσεων ενεργοποίησης Relu και Softmax καθώς και την παράγωγο της Relu. Η συνάρτηση ενεργοποίησης Relu ορίζεται ως $f(x) = \max(0, x)$. Είναι μη γραμμική και χρησιμοποιείται στα κρυφά επίπεδα του δικτύου κατά την διαδικασία του forward propagation, και συγκεκριμένα εφαρμόζεται στις εξόδους z κάθε κρυφού επιπέδου, ώστε να εισάγει μη γραμμικότητα στο δίκτυο. Αυτό επιτρέπει στο νευρωνικό δίκτυο να μάθει πιο πολύπλοκες σχέσεις μεταξύ των δεδομένων

εισόδου και εξόδου. Η παράγωγος της ReLU ορίζεται ως $f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$ και χρησιμοποιείται κατά

τη διάρκεια της διαδικασίας backward propagation για την ενημέρωση των βαρών. Η συνάρτηση Softmax ορίζεται ως $f_i(x) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$ όπου:

- z_i : η είσοδος στον i -οστό νευρώνα του τελευταίου επιπέδου.
- C : ο αριθμός των κατηγοριών.
- $f_i(x)$: η πιθανότητα η είσοδος να ανήκει στην κατηγορία i .

και μετατρέπει τις εξόδους του δικτύου σε πιθανότητες που αθροίζουν στο 1, επιτρέποντας τη χρήση της cross-entropy loss για την εκπαίδευση.

```

# συναρτηση ενεργοποίησης ReLU
# επιστρεφει x εαν x>0
# αλλιως επιστρεφει 0
def relu(x):
    return np.maximum(0, x)

# παραγωγος της ReLU
def relu_derivative(x):
    return (x > 0).astype(float)

# μετατροπη των εξοδων σε πιθανότητες
def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

```

Σχήμα 2: Συναρτήσεις Ενεργοποίησης.

2.4 Ορισμός συνάρτησης κόστους.

Μετά από τις 2 συναρτήσεις ενεργοποίησης, ορίζεται η `cross_entropy_loss` και η παραγωγός της `cross_entropy_derivative`. Η Cross Entropy είναι μια συνάρτηση κόστους (loss function), η οποία χρησιμοποιείται σε προβλήματα ταξινόμησης και έχει άμεση σχέση με την συνάρτηση Softmax στο τελικό επίπεδο εξόδου. Συγκεκριμένα, η Softmax παράγει μια κατανομή πιθανοτήτων για κάθε κατηγορία και η Cross Entropy συγκρίνει αυτή την κατανομή με την πραγματική κατανομή y_{true} . Η συνάρτηση Cross Entropy ορίζεται ως:

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$$

όπου:

- m : Ο αριθμός των δειγμάτων στο batch.
- C : Ο αριθμός των κατηγοριών.
- y_{ij} : Η πραγματική ετικέτα του i -οστού δείγματος για την κατηγορία j (σε μορφή **one-hot encoding**).
- $\hat{y}_{ij}$: Η προβλεπόμενη πιθανότητα (output του softmax) για το i -οστό δείγμα και την κατηγορία j .

Η Cross Entropy μετρά την απόσταση μεταξύ των προβλεπόμενων πιθανοτήτων ($\hat{y}$) και της πραγματικής κατανομής (y). Αν το δίκτυο προβλέπει σωστά την κατηγορία, δηλαδή $\hat{y}_j \approx 1$ για τη σωστή κατηγορία, τότε:

$$L \approx -\log(1) = 0$$

Επομένως, το κόστος είναι μικρό. Αν η πιθανότητα για τη σωστή κατηγορία είναι χαμηλή ($\hat{y}_j \approx 0$), το κόστος γίνεται μεγάλο:

$$L \approx -\log(0) \rightarrow \infty$$

Η παράγωγος της Cross Entropy υπολογίζεται σε συνδυασμό με την Softmax. Συγκεκριμένα, όπως προαναφέρθηκε, η συνάρτηση κόστους ισούται με:

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$$

Η παράγωγος της Cross Entropy ως προς την έξοδο του δικτύου $\hat{y}_{ij}$ δίνεται από:

$$\frac{\partial L}{\partial \hat{y}_{ij}} = \hat{y}_{ij} - y_{ij}$$

```
# υπολογισμός της αποκλίσης μεταξύ των πραγματικών (y_true) και προβλεπόμενων τιμών y_pred
# (υπολογισμός μεσού όρου loss για το batch)
# y_true.shape[0] αντιστοιχεί στον αριθμό των δειγμάτων m στο τρέχον (στην προκειμένη περίπτωση είναι 128)
def cross_entropy_loss(y_true, y_pred):
    return -np.sum(y_true * np.log(y_pred + 1e-8)) / y_true.shape[0]

# παραγωγός της cross entropy
def cross_entropy_derivative(y_true, y_pred):
    return y_pred - y_true
```

Σχήμα 3: Cross Entropy - Παράγωγος Cross Entropy

2.5 Κατασκευή batches

Η συνάρτηση `create_batches` είναι μια βοηθητική συνάρτηση που ανακατεύει τα δεδομένα εισόδου και τα χωρίζει σε μικρότερα τμήματα mini batches. Η τυχαία αναδιάταξη των δειγμάτων διασφαλίζει ότι κάθε batch περιέχει διαφορετικό συνδυασμό δεδομένων σε κάθε εποχή, γεγονός που βοηθά στη μείωση της πιθανότητας υπερβολικής προσαρμογής (overfitting) και στη διασφάλιση ότι η εκπαίδευση δεν εξαρτάται από την αρχική σειρά των δεδομένων, δηλαδή εξασφαλίζει την γενίκευση. Το χώρισμα των δεδομένων σε μικρότερα batches εξασφαλίζει γρηγορότερη επεξεργασία και μικρότερες απαιτήσεις μνήμης.

```
# ανακατεύω τα δεδομένα για να εξασφαλίσω ότι το κάθε batch θα ναι τυχαίο
# διαχωρίζω τα δεδομένα σε μικρά batches
def create_batches(X, y, batch_size):
    # ανακατεύω τα δεδομένα και τα labels με την βοήθεια των τυχαίων δεικτών
    num_samples = X.shape[0]
    indices = np.arange(num_samples)
    np.random.shuffle(indices)
    X = X[indices]
    y = y[indices]

    # διαδικασία διαχωρισμού σε batches
    batches = []
    for i in range(0, num_samples, batch_size):
        X_batch = X[i:i+batch_size]
        y_batch = y[i:i+batch_size]
        batches.append((X_batch, y_batch))
    return batches
```

Σχήμα 4: Shuffling - Mini Batches.

2.6 Κατασκευή Νευρωνικού Δικτύου

Στην συνέχεια ορίζουμε την κλάση `NeuralNetwork`, η οποία αποτελεί την κύρια υλοποίηση του νευρωνικού δικτύου. Περιλαμβάνει τη δημιουργία της δομής του δικτύου, τη διαδικασία forward propagation, το backward propagation, καθώς και την εκπαίδευση και αξιολόγηση του μοντέλου.

Παίρνει σαν είσοδο το μέγεθος του διανύσματος εισόδου (3072) `input_size`, την λίστα με το πλήθος νευρώνων σε κάθε κρυφό επίπεδο (`hidden_sizes`), το πλήθος των κατηγοριών εξόδου (10) (`output_size`) και τον ρυθμός μάθησης για την ενημέρωση των βαρών `learning_rate`. Αρχικοποιούμε τα βάρη με He initialization ($W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{input size}}}\right)$) και τα biases στο 0. Έπειτα γίνεται ο υπολογισμός του forward propagation.

```

class NeuralNetwork:
    def __init__(self, input_size, hidden_sizes, output_size, learning_rate=0.01):
        self.learning_rate = learning_rate
        self.weights = []
        self.biases = []
        # αρχικοποίηση των βάρων με He initialization
        # αρχικοποίηση του bias στο 0
        layer_sizes = [input_size] + hidden_sizes + [output_size]
        for i in range(len(layer_sizes) - 1):
            self.weights.append(np.random.randn(layer_sizes[i], layer_sizes[i+1]) * np.sqrt(2 / layer_sizes[i]))
            self.biases.append(np.zeros((1, layer_sizes[i+1])))

```

Σχήμα 5: Αρχικοποίηση Βαρών - Bias

2.6.1 Forward Propagation

Η μέθοδος αυτή υπολογίζει τις εξόδους του δικτύου περνώντας τα δεδομένα εισόδου μέσα από τα επίπεδα.

- **Διαδικασία:**

1. Σε κάθε επίπεδο, υπολογίζονται:

$$z^{(l)} = \text{activations}^{(l-1)} \cdot W^{(l)} + b^{(l)}$$

όπου W είναι τα βάρη και b τα bias και $\text{activations}^{(l-1)}$ είναι οι έξοδοι του $(l-1)$ επιπέδου.

2. Οι έξοδοι περνούν από τις συναρτήσεις ενεργοποίησης:

- **ReLU** για τα κρυφά επίπεδα.
- **Softmax** για το τελικό επίπεδο, που παράγει πιθανότητες.

- **Επιστροφή:**

- Επιστρέφει την τελική έξοδο του δικτύου (πιθανότητες κατηγορίας).

```

# forward propagation
def forward(self, X):
    self.activations = [X]
    self.z_values = []

    for i in range(len(self.weights) - 1):
        z = np.dot(self.activations[-1], self.weights[i]) + self.biases[i] # υπολογισμός εξόδου
        self.z_values.append(z)
        self.activations.append(relu(z)) # ενεργοποίηση με ReLU

    z = np.dot(self.activations[-1], self.weights[-1]) + self.biases[-1]
    self.z_values.append(z)
    self.activations.append(softmax(z))
    return self.activations[-1]

```

Σχήμα 6: Forward Propagation

Έπειτα, ακολουθεί το Backward Propagation.

2.6.2 Backward Propagation

Η μέθοδος αυτή υπολογίζει το σφάλμα και τις παραγώγους για την ενημέρωση των βάρων.

- **Διαδικασία:**

1. Υπολογίζεται το σφάλμα εξόδου:

$$\delta = \hat{y} - y$$

2. Επιστρέφοντας στα προηγούμενα επίπεδα:

$$\delta^{(l)} = \left(\delta^{(l+1)} W^{(l+1)} \right) \cdot f'^{(l)}(z)$$

όπου $f'^{(l)}$ είναι η παράγωγος της ReLU.

- **Ενημέρωση Βαρών:**

$$W^{(l)} := W^{(l)} - \eta \cdot \left(\text{activations}^{(l-1)} \right)^T \cdot \delta^{(l)}$$

$$b^{(l)} := b^{(l)} - \eta \cdot \delta^{(l)}$$

```
# backward propagation
def backward(self, X, y_true):
    gradients_w = []
    gradients_b = []

    # υπολογισμός σφαλματος εξόδου
    delta = cross_entropy_derivative(y_true, self.activations[-1])
    for i in reversed(range(len(self.weights))):
        # παραγωγοί για weights και biases
        grad_w = np.dot(self.activations[i].T, delta) / X.shape[0]
        grad_b = np.sum(delta, axis=0, keepdims=True) / X.shape[0]
        gradients_w.insert(0, grad_w)
        gradients_b.insert(0, grad_b)

        # υπολογισμός σφαλματος για hidden layers
        if i != 0:
            delta = np.dot(delta, self.weights[i].T) * relu_derivative(self.z_values[i-1])

    for i in range(len(self.weights)):
        self.weights[i] -= self.learning_rate * gradients_w[i]
        self.biases[i] -= self.learning_rate * gradients_b[i]
```

Σχήμα 7: Backward Propagation

2.6.3 Εκπαίδευση Δικτύου

Η συνάρτηση `train` αποτελεί το βασικό μέρος της εκπαίδευσης του νευρωνικού δικτύου. Συνδυάζει τη δημιουργία mini batches, τη διαδικασία `forward propagation`, τη διαδικασία `backward propagation`, και την αξιολόγηση της απόδοσης σε κάθε εποχή. Δέχεται σαν είσοδο τα δεδομένα εκπαίδευσης και δοκιμής:

- `X_train, y_train`: Τα δεδομένα εκπαίδευσης και οι ετικέτες τους.
- `X_test, y_test`: Τα δεδομένα δοκιμής και οι ετικέτες τους.
- `epochs`: Ο αριθμός των εποχών.
- `batch_size`: Το μέγεθος του κάθε mini batch.

και αρχικοποιεί τις μεταβλητές για την παρακολούθηση της καλύτερης απόδοσης:

- `best_epoch`: Αποθηκεύει την εποχή με την υψηλότερη ακρίβεια στο test set.
- `best_accuracy`: Καταγράφει τη μέγιστη ακρίβεια στο test set.
- `best_stats`: Αποθηκεύει τα στατιστικά της καλύτερης εποχής (π.χ., train loss, test loss, ακρίβεια).

Σε έναν βρόχο που εκτελείται για τον αριθμό των εποχών, δημιουργεί τα mini batches και για κάθε mini batch εκτελεί `forward propagation` υπολογίζοντας έτσι την έξοδο του νευρωνικού δικτύου, υπολογίζει το loss με την συνάρτηση `cross entropy` και υπολογίζει την ακρίβεια, συγκρίνοντας τις προβλέψεις του μοντέλου με τις πραγματικές ετικέτες. Έπειτα, μέσω του `backward propagation` υπολογίζονται οι παράγωγοι και ενημερώνονται τα βάρη και τα biases, ολοκληρώνοντας έτσι της εκπαίδευση.

Αφού ολοκληρωθεί η εκπαίδευση, αξιολογείται η απόδοση στο test set. Αν η ακρίβεια στο test set είναι η υψηλότερη που έχει καταγραφεί, ενημερώνονται τα καλύτερα στατιστικά.

Μετά από κάθε εποχή, εμφανίζονται τα στατιστικά της εκπαίδευσης και δοκιμής και τέλος, αφού ολοκληρωθούν όλες οι εποχές, εμφανίζονται τα στατιστικά της καλύτερης εποχής.

```
def train(self, X_train, y_train, X_test, y_test, epochs=10, batch_size=128):
    best_epoch = None # εποχή με το καλύτερο test accuracy
    best_accuracy = 0 # υψηλότερη ακρίβεια στο test set
    best_stats = {} # στατιστικά της καλύτερης εποχής (train loss, test loss, train accuracy, test accuracy)

    start_time = time.time() # Έναρξη συνολικού χρόνου εκτέλεσης

    for epoch in range(epochs):
        train_batches = create_batches(X_train, y_train, batch_size)

        # train_loss: συνολικό loss για όλα τα mini-batches
        # train_correct: το πλήθος των σωστών προβλέψεων
        train_loss, train_correct = 0, 0
        for X_batch, y_batch in train_batches:
            predictions = self.forward(X_batch) # υπολογισμός προβλεψεων νευρωνικού
            train_loss += cross_entropy_loss(y_batch, predictions) # loss για το τρεχον batch
            train_correct += np.sum(np.argmax(predictions, axis=1) == np.argmax(y_batch, axis=1)) # υπολογισμός ακριβειας του
            # batch συγκρινοντας τις προβλεψεις με τις πραγματικες ετικετες
        self.backward(X_batch, y_batch) # υπολογισμός και ενημερωση βαρων

        test_predictions = self.forward(X_test) # υπολογισμός προβλεψεων για τα δεδομένα δοκιμής
        test_loss = cross_entropy_loss(y_test, test_predictions) # υπολογισμός συνολικού loss στο test set
        test_accuracy = np.mean(np.argmax(test_predictions, axis=1) == np.argmax(y_test, axis=1))

        train_loss /= len(train_batches) # μέσος όρος του loss για όλα τα batches
        train_accuracy = train_correct / X_train.shape[0] # υπολογισμός ακριβειας

        # ενημερωση των καλύτερων αποτελεσμάτων
        if test_accuracy > best_accuracy:
            best_accuracy = test_accuracy
            best_epoch = epoch + 1
            best_stats = {
                "train_loss": train_loss,
                "test_loss": test_loss,
                "train_accuracy": train_accuracy * 100,
                "test_accuracy": test_accuracy * 100,
            }

    total_time = time.time() - start_time # Συνολικός χρόνος εκτέλεσης μέχρι τώρα
```

Σχήμα 8: Train 1

```
# εμφάνιση αποτελεσμάτων
print(f"Epoch {epoch + 1}/{epochs}")
print(f"Training Loss: {train_loss:.4f}, Training Accuracy: {train_accuracy * 100:.2f}%")
print(f"Testing Loss: {test_loss:.4f}, Testing Accuracy: {test_accuracy * 100:.2f}%")
print(f"Total time until epoch {epoch + 1}: {total_time:.2f} seconds")
print("-" * 50)

# τελική εμφάνιση καλύτερων αποτελεσμάτων
print("\nBest Results:")
print(f"Epoch: {best_epoch}")
print(f"Training Loss: {best_stats['train_loss']:.4f}, Training Accuracy: {best_stats['train_accuracy']:.2f}%")
print(f"Testing Loss: {best_stats['test_loss']:.4f}, Testing Accuracy: {best_stats['test_accuracy']:.2f}%")
```

Σχήμα 9: Train 2

2.6.4 Αξιολόγηση απόδοσης

Η συνάρτηση evaluate χρησιμοποιείται για την αξιολόγηση της απόδοσης του εκπαιδευμένου νευρωνικού δικτύου. Παίρνει ως είσοδο ένα σύνολο δεδομένων X και τις αντίστοιχες ετικέτες y . Η μέθοδος forward περνά τα δεδομένα εισόδου X μέσα από το δίκτυο, παράγει τις πιθανότητες κατηγορίας για κάθε δείγμα στο σύνολο X και εντοπίζει τον δείκτη της μέγιστης πιθανότητας για κάθε δείγμα, ο οποίος δείκτης αυτός αντιστοιχεί στην κατηγορία που το δίκτυο προβλέπει. Έπειτα, συγκρίνει τις προβλέψεις του δικτύου με τις πραγματικές ετικέτες και υπολογίζει τον μέσο όρο των σωστών προβλέψεων. Τέλος, επιστρέφει την ακρίβεια ως δεκαδικό ποσοστό και την εκτυπώνει σε ποσοστό.

```
def evaluate(self, X, y):
    predictions = np.argmax(self.forward(X), axis=1) # υπολογίζω τις προβλέψεις του δικτύου y μέσω της μεθόδου forward
    accuracy = np.mean(predictions == np.argmax(y, axis=1)) # συγκρίνω τις προβλέψεις με τις
    # πραγματικές ετικέτες y για τον υπολογισμό της ακριβειας
    print(f"Accuracy: {accuracy * 100:.2f}%")
```

Σχήμα 10: Evaluate

2.7 One-hot encoding

Η συνάρτηση `preprocess_data` μετατρέπει τις ετικέτες των δεδομένων εκπαίδευσης και δοκιμής (labels) από ακέραιες τιμές (π.χ. 0-9) σε one-hot encoding. Με την χρήση του one-hot encoding αναπαριστούμε κάθε κατηγορία με ένα διάνυσμα μήκους ίσο με το πλήθος των κατηγοριών, στα οποία διανύσματα η τιμή 1 τοποθετείται στην θέση που αντιστοιχεί στην κατηγορία, ενώ όλες οι υπόλοιπες τιμές είναι 0. Στο δίκτυο για το οποίο χρησιμοποιούμε την βάση δεδομένων Cifar-10, περιέχονται 10 κατηγορίες και οι ετικέτες παρέχονται αρχικά ως ακέραιοι αριθμοί: [0, 1, ..., 9]. Αν η ετικέτα μιας εικόνας είναι 3, το αντίστοιχο one-hot encoded διάνυσμα είναι: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. Η έξοδος του δικτύου (πιθανότητες μέσω softmax) συγκρίνεται με το one-hot encoded διάνυσμα κατά τη συνάρτηση κόστους (cross-entropy loss).

Για την υλοποίηση του one-hot encoding στο κώδικα μας, χρησιμοποιούμε την συνάρτηση `np.eye(10)`, η οποία δημιουργεί έναν πίνακα ταυτότητας 10×10 , όπου κάθε γραμμή είναι ένα one-hot encoded διάνυσμα. Το `train_labels` και το `test_labels` περιέχουν τις ετικέτες ως αριθμούς (π.χ., 0-9). Το `np.eye(10)[train_labels]` και το `np.eye(10)[test_labels]` επιλέγουν τις σωστές γραμμές από τον πίνακα ταυτότητας.

```
# μετατροπή ετικετων σε one-hot encoding
def preprocess_data(train_data, train_labels, test_data, test_labels):
    train_labels_one_hot = np.eye(10)[train_labels] # 10*10 identity matrix
    test_labels_one_hot = np.eye(10)[test_labels]
    return train_data, train_labels_one_hot, test_data, test_labels_one_hot
```

Σχήμα 11: One-hot encoding

2.8 Κύριο Μέρος Κώδικα

Περιέχεται η βασική ροή για τη φόρτωση δεδομένων, τη δημιουργία του δικτύου, την εκπαίδευση, και την αξιολόγηση.

Αρχικά γίνεται η έναρξη της μέτρησης χρόνου για να υπολογιστεί η συνολική διάρκεια εκτέλεσης του προγράμματος.

Έπειτα γίνεται η φόρτωση και η προεπεξεργασία των δεδομένων. Συγκεκριμένα, 'φορτώνονται' τα δεδομένα Cifar-10 και μετατρέπονται οι ετικέτες σε μορφή one-hot encoding.

Στην συνέχεια υπολογίζει και αποθηκεύει στην μεταβλητή `input_size` τον αριθμό των χαρακτηριστικών κάθε δείγματος (3072 pixels), καθορίζει και αποθηκεύει στην μεταβλητή `hidden_sizes` τον αριθμό νευρώνων σε κάθε κρυφό επίπεδο και στην μεταβλητή `output_size` δηλώνεται ο αριθμός των εξόδων του δικτύου, ο οποίος ισούται με το πλήθος των κατηγοριών στο Cifar-10.

Ακολουθεί η δημιουργία του νευρωνικού δικτύου καλώντας την κλάση `NeuralNetwork`, η εκπαίδευση του δικτύου με την συνάρτηση `train` και την αξιολόγηση του δικτύου στο test-set.

Τέλος, υπολογίζεται και εμφανίζεται η συνολική διάρκεια εκτέλεσης του προγράμματος.

```
if __name__ == "__main__":
    start_time = time.time()

    data_dir = os.path.expanduser("~/Desktop/Neural_Networks_project")
    train_data, train_labels, test_data, test_labels = load_cifar10_data(data_dir)
    train_data, train_labels, test_data, test_labels = preprocess_data(train_data, train_labels, test_data, test_labels)

    # ορισμός δικτύου
    input_size = train_data.shape[1] # αριθμός pixel (3072)
    hidden_sizes = [512, 256, 128]
    output_size = 10

    nn = NeuralNetwork(input_size, hidden_sizes, output_size, learning_rate=0.01)
    nn.train(train_data, train_labels, test_data, test_labels, epochs=300, batch_size=128)
    nn.evaluate(test_data, test_labels)

    end_time = time.time()
    print(f"\nTotal Execution Time: {end_time - start_time:.2f} seconds")
```

Σχήμα 12: One-hot encoding

3 Δοκιμές

Στα πλαίσια των δοκιμών του νευρωνικού δικτύου, πραγματοποιήθηκαν 2 είδη δοκιμών: Για σταθερό learning rate, και για μεταβλητό learning rate.

Learning rate: Είναι ένας συντελεστής κλιμάκωσης που καθορίζει το μέγεθος του βήματος που θα κάνει ο αλγόριθμος βελτιστοποίησης για την ενημέρωση των βαρών του δικτύου κατά τη διάρκεια της εκπαίδευσης. Για υψηλό learning rate, το μοντέλο μπορεί να κάνει μεγάλα βήματα προς το βέλτιστο, αλλά υπάρχει κίνδυνος να υπερπηδήσει το ελάχιστο της συνάρτησης κόστους, καθώς επίσης να είναι ασταθές και να μην συγκλίνει. Για χαμηλό learning rate το μοντέλο κάνει μικρά βήματα, κάτι που εξασφαλίζει σταθερότερη εκπαίδευση. Ωστόσο, η εκπαίδευση μπορεί να είναι πολύ αργή.

Αξίζει να δωθεί σε αυτό το σημείο ο ορισμός του overfitting, ενός φαινομένου που συναντάται σε όλες τις δοκιμές μας.

Overfitting: Το μοντέλο “μαθαίνει πολύ καλά” τα δεδομένα εκπαίδευσης, σε σημείο που αποτυγχάνει να γενικεύσει καλά σε νέα, άγνωστα δεδομένα test set. Αυτό σημαίνει ότι το μοντέλο αποδίδει εξαιρετικά στα δεδομένα εκπαίδευσης, αλλά έχει χαμηλή ακρίβεια ή υψηλό σφάλμα στα δεδομένα δοκιμής. Παρατηρούμε το overfitting στις δοκιμές μας όταν το training loss συνεχίζει να μειώνεται ενώ το testing loss σταματά την μείωση του και αρχίζει να αυξάνεται, καθώς επίσης όταν παρατηρούμε ότι το training accuracy συνεχίζει να αυξάνεται ενώ το testing accuracy μετά την επίτευξη του μέγιστού του, αρχίζει να μειώνεται.

3.1 Σταθερό learning rate

Οι τιμές που δοκιμάστηκαν για το learning rate είναι:

- $\eta=0.1$
- $\eta=0.01$
- $\eta=0.001$

3.1.1 1 hidden layer

Στην αρχή δοκιμάστηκε η χρήση ενός κρυφού επιπέδου, για διαφορετικό αριθμό νευρώνων και learning rate. Πειραματιζόμαστε για **64, 128, 256, 512** αριθμούς νευρώνων.

- **64 νευρώνες - $\eta = 0.1$**

Για 64 νευρώνες και learning rate=0.1 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **112.29 δευτερόλεπτα** στην εποχή **165/500** και είναι **47.50%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 53.38%**, **Training Loss: 1.3006** και **Testing Loss: 1.5337**. Παρατηρούμε ότι μετά την εποχή που σημειώνεται το υψηλότερο testing accuracy, δημιουργείται το φαινόμενο του overfitting. Όπως θα φανεί και στον πίνακα με κάποια δείγματα εποχών, για το training loss και το testing loss, μέχρι να επιτευχθεί το υψηλότερο testing accuracy και οι 2 τιμές σταδιακά μειώνονται, ενώ μετά την επίτευξη της υψηλότερης τιμής testing accuracy, το training loss συνεχίζει να μειώνεται, ενώ το testing accuracy αρχίζει να αυξάνεται. Επίσης, παρατηρούμε ότι το training accuracy μονίμως αυξάνεται, εν αντιθέση με το testing accuracy, το οποίο μετά την επίτευξη της μέγιστης τιμής του, αρχίζει να μειώνεται.

Παρακάτω βλέπουμε έναν πίνακα με κάποια δείγματα εποχών, θέλοντας έτσι να δείξουμε το φαινόμενο του overfitting.

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.1443	18.46%	1.9971	23.93%
40	1.5221	45.36%	1.5780	44.15%
100	1.3769	50.78%	1.5564	45.01%
165	1.3006	53.38%	1.5337	47.50%
200	1.2772	54.37%	1.5564	45.61%
402	1.1646	57.98%	1.7520	43.49%
500	1.1365	59.17%	1.9410	40.54%

Πίνακας 1: Δείγματα Εποχών.

Epoch 500/500

Training Loss: 1.1365, Training Accuracy: 59.17%

Testing Loss: 1.9410, Testing Accuracy: 40.54%

Total time until epoch 500: 347.53 seconds

Best Results:

Epoch: 165

Training Loss: 1.3006, Training Accuracy: 53.38%

Testing Loss: 1.5337, Testing Accuracy: 47.50%

Accuracy: 40.54%

Σχήμα 13: 1 hidden layer with 64 neuron and learning rate 0.1

• 64 νευρώνες - $\eta = 0.01$

Για 64 νευρώνες και learning rate=0.01 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **108.44 δευτερόλεπτα** στην εποχή **144/500** και είναι **51.05%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 59.55%**, **Training Loss: 1.1540** και **Testing Loss: 1.4173**. Όπως και στην προηγούμενη δοκιμή, έτσι και εδώ, παρατηρούμε ότι από την εποχή 144 και μετά ξεκινάει **overfitting**. Ακολουθεί πίνακας με δείγματα εποχών

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9983	28.02%	1.8804	34.10%
40	1.3874	51.62%	1.4630	48.17%
100	1.2265	57.24%	1.4121	50.29%
144	1.1540	59.55%	1.4173	51.05%
200	1.0813	62.35%	1.4305	50.20%
402	0.9133	67.83%	1.6253	48.43%
500	0.8554	69.85%	1.7965	45.51%

Πίνακας 2: Δείγματα Εποχών.

ΣΗΜΕΙΩΣΗ

Είναι φανερό ότι για 64 νευρώνες, το μοντέλο με learning rate 0.01 είναι ανώτερο από αυτό με learning rate 0.01 διότι το μέγιστο Test accuracy είναι μεγαλύτερο στο πρώτο μοντέλο και επιτυγχάνεται ταχύτερα.

```
Epoch 500/500
Training Loss: 0.8554, Training Accuracy: 69.85%
Testing Loss: 1.7965, Testing Accuracy: 45.51%
Total time until epoch 500: 366.60 seconds
```

```
Best Results:
Epoch: 144
Training Loss: 1.1540, Training Accuracy: 59.55%
Testing Loss: 1.4173, Testing Accuracy: 51.05%
Accuracy: 45.51%
```

Σχήμα 14: 1 hidden layer with 64 neuron and learning rate 0.01

- 64 νευρώνες - $\eta = 0.001$

Για 64 νευρώνες και learning rate=0.001 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **730.21 δευτερόλεπτα** στην εποχή **1057/1200** και είναι **50.92%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 61.16%**, **Training Loss: 1.1215** και **Testing Loss: 1.4129**. Στην προκείμενη περίπτωση, παρόλο που δεν μεσολαβούν πολλές εποχές μέχρι την ολοκλήρωση των εποχών, είναι πάλι φανερό ότι ξεκινάει overfitting μετά την εποχή 1057, από τις τιμές των testing accuracy και training accuracy.

Ακολουθεί πίνακας με δείγματα εποχών:

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.2108	19.47%	2.1288	24.49%
40	1.6980	41.32%	1.7085	41.00%
100	1.5751	45.81%	1.6011	44.14%
200	1.4591	49.72%	1.5120	47.25%
500	1.2869	55.42%	1.4264	49.48%
1057	1.1215	61.16%	1.4129	50.92%
1100	1.1114	61.64%	1.4151	50.33%
1200	1.0899	62.44%	1.4175	50.32%

Πίνακας 3: Δείγματα Εποχών.

```
Epoch 1200/1200
Training Loss: 1.0899, Training Accuracy: 62.44%
Testing Loss: 1.4175, Testing Accuracy: 50.60%
Total time until epoch 1200: 831.82 seconds
```

```
Best Results:
Epoch: 1057
Training Loss: 1.1215, Training Accuracy: 61.16%
Testing Loss: 1.4129, Testing Accuracy: 50.92%
Accuracy: 50.60%
```

Σχήμα 15: 1 hidden layer with 64 neuron and learning rate 0.001

Παρατηρούμε ότι το μέγιστο training accuracy για learning rate 0.001 είναι πολύ καλύτερο από την περίπτωση το μοντέλο για learning rate 0.1 και ελαφρώς χειρότερο από το μοντέλο για learning rate 0.01. Ωστόσο παρατηρούμε ότι απαιτείται πολύς χρόνος για να επιτευχθεί το μέγιστο training accuracy πράγμα που καθιστά το μοντέλο ασύμφορο και μας παρακινεί να χρησιμοποιήσουμε το μοντέλο για learning rate 0.01.

- **128 νευρώνες - $\eta = 0.1$**

Για 128 νευρώνες και learning rate=0.1 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **63.26 δευτερόλεπτα** στην εποχή **66/500** και είναι **49.60%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 57.21%**, **Training Loss: 1.2072** και **Testing Loss: 1.4886**. Η παρατήρηση για το overfitting ισχύει και σε αυτή τη περίπτωση.

Σε αυτή τη περίπτωση βλέπουμε πως για το ίδιο learning rate (0.1) η αύξηση των νευρώνων στο hidden layer από 64 σε 128, αύξησε αρκετά το testing accuracy, μειώνοντας στο μισό τον χρόνο που απαιτείται για να επιτευχθεί. Ακολουθεί πίνακας με δείγματα εποχών.

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0104	26.22%	1.8487	33.15%
40	1.3230	53.44%	1.5279	46.47%
66	1.2072	57.21%	1.4886	49.60%
100	1.0988	60.78%	1.7067	46.10%
200	0.8588	69.44%	1.9365	46.01%
300	0.7069	74.93%	2.2910	45.32%
500	0.5148	81.82%	3.1130	44.75%

Πίνακας 4: Δείγματα Εποχών.

```
Epoch 500/500
Training Loss: 0.5148, Training Accuracy: 81.82%
Testing Loss: 3.1130, Testing Accuracy: 44.75%
Total time until epoch 500: 473.02 seconds
-----
Best Results:
Epoch: 66
Training Loss: 1.2072, Training Accuracy: 57.21%
Testing Loss: 1.4886, Testing Accuracy: 49.60%
Accuracy: 44.75%
```

Σχήμα 16: 1 hidden layer with 128 neuron and learning rate 0.1

- **128 νευρώνες - $\eta = 0.01$**

Για 128 νευρώνες και learning rate=0.01 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **472.08 δευτερόλεπτα** στην εποχή **189/500** και είναι **52.64%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 67.87%**, **Training Loss: 0.9349** και **Testing Loss: 1.4198**. Η παρατήρηση για το overfitting ισχύει και σε αυτή τη περίπτωση.

Παρατηρείται μια αύξηση του testing accuracy σε σχέση με το μοντέλο που χρησιμοποιεί 64 νευρώνες με το ίδιο learning rate, ωστόσο απαιτείται σημαντικά περισσότερος χρόνος για να επιτευχθεί αυτό το βελτιωμένο testing accuracy.

Ακολουθεί πίνακας με δείγματα εποχών.

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9931	28.75%	1.8691	34.17%
40	1.3359	53.32%	1.4351	49.08%
100	1.1206	61.19%	1.4256	50.45%
189	0.9349	67.87%	1.4198	52.64%
300	0.7888	72.78%	1.5259	51.38%
500	0.5793	80.22%	1.8201	50.29%

Πίνακας 5: Δείγματα Εποχών.

```

Epoch 500/500
Training Loss: 0.5793, Training Accuracy: 80.22%
Testing Loss: 1.8201, Testing Accuracy: 50.29%
Total time until epoch 500: 471.68 seconds
-----

Best Results:
Epoch: 189
Training Loss: 0.9349, Training Accuracy: 67.87%
Testing Loss: 1.4198, Testing Accuracy: 52.64%
Accuracy: 50.29%

```

Σχήμα 17: 1 hidden layer with 128 neuron and learning rate 0.01

- 128 νευρώνες - $\eta = 0.001$

Για 128 νευρώνες και learning rate=0.001 λαμβάνουμε τα εξής αποτελέσματα:

Το καλύτερο **Test accuracy** παρατηρείται μετά από **1013.98 δευτερόλεπτα** στην εποχή **1047/500** και είναι **52.99%**. Επίσης γι' αυτή την εποχή έχουμε **Training Accuracy: 65.61%**, **Training Loss: 1.0143** και **Testing Loss: 1.3670**. Η παρατήρηση για το overfitting ισχύει και σε αυτή τη περίπτωση.

Μπορούμε να παρατηρήσουμε ότι σε αυτή τη περίπτωση σημειώνεται το υψηλότερο testing accuracy μέχρι στιγμής. Ωστόσο, ο χρόνος που απαιτείται για να επιτευχθεί είναι ο υψηλότερος που έχουμε συναντήσει.

Ακολουθεί πίνακας με δείγματα εποχών.

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.1983	19.64%	2.1209	23.76%
40	1.6910	41.50%	1.6993	40.67%
100	1.5565	46.36%	1.5837	44.69%
500	1.2305	57.69%	1.3879	51.51%
1000	1.0291	65.08%	1.3705	52.47%
1047	1.0143	65.61%	1.3670	52.99%
1200	0.9669	67.39%	1.3748	52.27%

Πίνακας 6: Δείγματα Εποχών.

```

Epoch 1200/1200
Training Loss: 0.9669, Training Accuracy: 67.39%
Testing Loss: 1.3748, Testing Accuracy: 52.27%
Total time until epoch 1200: 1157.22 seconds
-----

Best Results:
Epoch: 1047
Training Loss: 1.0143, Training Accuracy: 65.61%
Testing Loss: 1.3670, Testing Accuracy: 52.99%
Accuracy: 52.27%

```

Σχήμα 18: 1 hidden layer with 128 neuron and learning rate 0.001

* Ακολουθούν οι πίνακες με τα δείγματα εποχών για 256 και 512 νευρώνες.

- 256 νευρώνες - $\eta = 0.1$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **181.90 δευτερόλεπτα** στην εποχή 98/500 και ισούται με **testing accuracy=51.40%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0124	27.32%	1.9720	29.57%
40	1.2156	57.08%	1.4977	48.25%
70	1.0314	63.63%	1.5271	50.46%
98	0.8981	68.26%	1.6180	51.40%
353	0.3380	88.39%	3.1327	48.65%
500	0.2088	92.94%	3.6852	48.31%

Πίνακας 7: Δείγματα Εποχών.

```

Epoch 500/500
Training Loss: 0.2088, Training Accuracy: 92.94%
Testing Loss: 3.6852, Testing Accuracy: 48.31%
Total time until epoch 500: 940.97 seconds
-----

Best Results:
Epoch: 98
Training Loss: 0.8981, Training Accuracy: 68.26%
Testing Loss: 1.6180, Testing Accuracy: 51.40%
Accuracy: 48.31%

```

Σχήμα 19: 1 hidden layer with 256 neuron and learning rate 0.01

- 256 νευρώνες - $\eta = 0.01$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **381.41 δευτερόλεπτα** στην εποχή 208/500 και ισούται με **testing accuracy=54.10%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9719	29.79%	1.8665	34.40%
50	1.2378	57.25%	1.3981	50.32%
100	1.0360	64.27%	1.3378	53.47%
208	0.7454	74.85%	1.4101	54.10%
350	0.5057	83.38%	1.9668	49.68%
500	0.3220	90.18%	2.0131	47.37%

Πίνακας 8: Δείγματα Εποχών.

```
Epoch 500/500
Training Loss: 0.3220, Training Accuracy: 90.18%
Testing Loss: 2.0131, Testing Accuracy: 47.37%
Total time until epoch 500: 928.31 seconds
-----

Best Results:
Epoch: 208
Training Loss: 0.7454, Training Accuracy: 74.85%
Testing Loss: 1.4101, Testing Accuracy: 54.10%
Accuracy: 52.09%
```

Σχήμα 20: 1 hidden layer with 256 neuron and learning rate 0.01

- 256 νευρώνες - $\eta = 0.001$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **2011.86 δευτερόλεπτα** στην εποχή 1072/1200 και ισούται με **testing accuracy=54.39%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.1940	20.04%	2.1039	25.93%
100	1.5285	47.63%	1.5571	45.80%
500	1.1710	60.31%	1.3515	52.61%
1000	0.9319	69.09%	1.3369	53.95%
1072	0.9020	70.32%	1.3430	54.39%
1200	0.8520	72.30%	1.3545	53.71%

Πίνακας 9: Δείγματα Εποχών.

```
Epoch 1200/1200
Training Loss: 0.8520, Training Accuracy: 72.30%
Testing Loss: 1.3545, Testing Accuracy: 53.71%
Total time until epoch 1200: 2256.44 seconds
-----

Best Results:
Epoch: 1072
Training Loss: 0.9020, Training Accuracy: 70.32%
Testing Loss: 1.3430, Testing Accuracy: 54.39%
Accuracy: 53.71%
```

Σχήμα 21: 1 hidden layer with 256 neuron and learning rate 0.001

- 512 νευρώνες - $\eta = 0.1$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **300.33 δευτερόλεπτα** στην εποχή 92/500 και ισούται με **testing accuracy=52.81%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0054	28.45%	1.8103	35.83%
50	1.0056	64.93%	1.5928	48.05%
70	0.8343	70.66%	1.6910	49.40%
92	0.6912	75.64%	1.7274	52.81%
200	0.2745	90.61%	2.7823	50.95%
500	0.0621	98.47%	4.0133	50.16%

Πίνακας 10: Δείγματα Εποχών.

Epoch 500/500

Training Loss: 0.0621, Training Accuracy: 98.47%

Testing Loss: 4.0133, Testing Accuracy: 50.16%

Total time until epoch 500: 1661.94 seconds

Best Results:

Epoch: 92

Training Loss: 0.6912, Training Accuracy: 75.64%

Testing Loss: 1.7274, Testing Accuracy: 52.81%

Accuracy: 50.16%

Σχήμα 22: 1 hidden layer with 512 neuron and learning rate 0.1

- 512 νευρώνες - $\eta = 0.01$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **619.06 δευτερόλεπτα** στην εποχή 190/500 και ισούται με **testing accuracy=54.82%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9633	30.02%	1.8656	32.79%
50	1.2033	58.44%	1.3523	52.47%
100	0.9743	66.84%	1.3249	53.31%
190	0.6694	78.01%	1.3822	54.82%
350	0.3075	91.38%	1.6806	54.15%
500	0.1500	97.13%	2.0570	52.99%

Πίνακας 11: Δείγματα Εποχών.

Epoch 500/500

Training Loss: 0.1500, Training Accuracy: 97.13%

Testing Loss: 2.0570, Testing Accuracy: 52.99%

Total time until epoch 500: 1654.36 seconds

Best Results:

Epoch: 190

Training Loss: 0.6694, Training Accuracy: 78.01%

Testing Loss: 1.3822, Testing Accuracy: 54.82%

Accuracy: 52.99%

Σχήμα 23: 1 hidden layer with 512 neuron and learning rate 0.01

- 512 νευρώνες - $\eta = 0.001$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **4522.12** δευτερόλεπτα στην εποχή 1331/1500 και ισούται με **testing accuracy=55.23%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.1892	20.56%	2.0947	25.14%
100	1.5248	47.88%	1.5550	46.34%
500	1.1417	61.28%	1.3346	52.86%
1000	0.8601	72.20%	1.3092	54.52%
1331	0.7093	78.37%	1.3227	55.23%
1400	0.6785	79.68%	1.3326	55.04%
1500	0.6373	81.19%	1.3429	54.01%

Πίνακας 12: Δείγματα Εποχών.

Epoch 1500/1500

Training Loss: 0.6373, Training Accuracy: 81.19%

Testing Loss: 1.3429, Testing Accuracy: 54.01%

Total time until epoch 1500: 5259.42 seconds

Best Results:

Epoch: 1331

Training Loss: 0.7093, Training Accuracy: 78.37%

Testing Loss: 1.3227, Testing Accuracy: 55.23%

Accuracy: 55.06%

Σχήμα 24: 1 hidden layer with 512 neuron and learning rate 0.001

ΣΥΜΠΕΡΑΣΜΑ: Το συμπέρασμα που εξάγουμε από όλα τα πειράματα για 1 hidden layer είναι ότι όσο μειώνεται το learning rate, έχουμε κατά βάση καλύτερο testing accuracy το οποίο όμως επιτυγχάνεται μετά από αρκετά περισσότερο χρόνο σε σχέση με τις περιπτώσεις που χρησιμοποιούμε μεγαλύτερο learning rate. Αυτό συμβαίνει διότι με μικρότερο learning rate το μοντέλο κάνει μικρότερα βήματα και αυτό σημαίνει ότι το μοντέλο μπορεί να προσαρμοστεί καλύτερα και να πλησιάσει πιο κοντά στο ελάχιστο της συνάρτησης κόστους. Αν το learning rate είναι πολύ μεγάλο το μοντέλο μπορεί να υπερπηδήσει το βέλτιστο σημείο, με αποτέλεσμα ασταθείς ενημερώσεις.

3.1.2 2 hidden layers

- 128, 64 νευρώνες - $\eta = 0.1$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **47.19** δευτερόλεπτα στην εποχή 42/500 και ισούται με **testing accuracy=51.03%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0197	26.44%	1.8436	33.70%
20	1.4017	50.00%	1.5189	45.86%
42	1.2322	56.00%	1.4012	51.03%
100	0.9713	65.14%	1.5832	50.06%
250	0.6148	77.75%	2.4716	47.76%
500	0.3751	86.68%	3.8042	43.99%

Πίνακας 13: Δείγματα Εποχών.

Epoch 500/500
 Training Loss: 0.3751, Training Accuracy: 86.68%
 Testing Loss: 3.8042, Testing Accuracy: 43.99%
 Total time until epoch 500: 583.16 seconds

Best Results:
 Epoch: 42
 Training Loss: 1.2322, Training Accuracy: 56.00%
 Testing Loss: 1.4012, Testing Accuracy: 51.03%
 Accuracy: 43.99%

Σχήμα 25: 2 hidden layers with 128 and 64 neurons and learning rate 0.1

• 128, 64 νευρώνες - $\eta = 0.01$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **123.58 δευτερόλεπτα** στην εποχή 110/500 και ισούται με **testing accuracy=53.11%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9987	27.92%	1.8897	30.92%
60	1.1749	58.68%	1.3955	51.22%
100	1.0285	63.86%	1.3813	51.91%
110	0.9975	65.14%	1.3803	53.11%
250	0.6655	76.70%	1.7710	49.24%
500	0.3258	89.44%	2.7661	48.57%

Πίνακας 14: Δείγματα Εποχών.

Epoch 500/500
 Training Loss: 0.3258, Training Accuracy: 89.44%
 Testing Loss: 2.7661, Testing Accuracy: 48.57%
 Total time until epoch 500: 564.23 seconds

Best Results:
 Epoch: 110
 Training Loss: 0.9975, Training Accuracy: 65.14%
 Testing Loss: 1.3803, Testing Accuracy: 53.11%
 Accuracy: 48.57%

Σχήμα 26: 2 hidden layers with 128 and 64 neurons and learning rate 0.01

• 128, 64 νευρώνες - $\eta = 0.001$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **516.77 δευτερόλεπτα** στην εποχή 426/1000 και ισούται με **testing accuracy=52.25%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.2029	18.97%	2.1276	23.15%
100	1.4917	48.21%	1.5210	46.32%
300	1.2330	56.98%	1.3870	51.15%
426	1.1266	60.68%	1.3722	52.25%
700	0.9452	67.31%	1.4204	51.81%
1000	0.7877	72.86%	1.5535	50.90%

Πίνακας 15: Δείγματα Εποχών.

Epoch 1000/1000
Training Loss: 0.7877, Training Accuracy: 72.86%
Testing Loss: 1.5535, Testing Accuracy: 50.90%
Total time until epoch 1000: 1197.52 seconds

Best Results:
Epoch: 426
Training Loss: 1.1266, Training Accuracy: 60.68%
Testing Loss: 1.3722, Testing Accuracy: 52.25%
Accuracy: 50.90%

Σχήμα 27: 2 hidden layers with 128 and 64 neurons and learning rate 0.001

- 256, 128 νευρώνες - $\eta = 0.1$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **98.73 δευτερόλεπτα** στην εποχή 48/500 και ισούται με **testing accuracy=51.74%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0631	26.01%	2.0464	27.78%
20	1.3346	52.34%	1.5915	46.19%
48	1.0734	61.43%	1.4381	51.74%
100	0.7398	73.61%	1.9553	48.53%
300	0.2226	92.02%	3.9829	48.12%
500	0.0350	99.05%	4.9518	48.78%

Πίνακας 16: Δείγματα Εποχών.

Epoch 500/500
Training Loss: 0.0350, Training Accuracy: 99.05%
Testing Loss: 4.9518, Testing Accuracy: 48.78%
Total time until epoch 500: 1044.52 seconds

Best Results:
Epoch: 48
Training Loss: 1.0734, Training Accuracy: 61.43%
Testing Loss: 1.4381, Testing Accuracy: 51.74%
Accuracy: 48.78%

Σχήμα 28: 2 hidden layers with 256 and 128 neurons and learning rate 0.1

- 256, 128 νευρώνες - $\eta = 0.01$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **162.48 δευτερόλεπτα** στην εποχή 79/500 και ισούται με **testing accuracy=53.86%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9807	29.40%	1.8538	34.34%
50	1.1427	60.01%	1.3574	52.59%
79	0.9771	65.76%	1.3249	53.86%
100	0.8647	69.81%	1.3705	53.55%
250	0.3194	89.81%	2.0174	52.34%
500	0.0187	99.91%	3.2930	52.19%

Πίνακας 17: Δείγματα Εποχών.

Epoch 500/500
 Training Loss: 0.0187, Training Accuracy: 99.91%
 Testing Loss: 3.2930, Testing Accuracy: 52.19%
 Total time until epoch 500: 1019.36 seconds

Best Results:
 Epoch: 79
 Training Loss: 0.9771, Training Accuracy: 65.76%
 Testing Loss: 1.3249, Testing Accuracy: 53.86%
 Accuracy: 52.19%

Σχήμα 29: 2 hidden layers with 256 and 128 neurons and learning rate 0.01

• 256, 128 νευρώνες - $\eta = 0.001$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **1210.46 δευτερόλεπτα** στην εποχή 620/500 και ισούται με **testing accuracy=53.95%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.1893	20.75%	2.1074	24.74%
100	1.4444	50.04%	1.4884	47.51%
300	1.1395	60.67%	1.3489	52.21%
620	0.8128	72.62%	1.3793	53.95%
800	0.6576	78.39%	1.5114	52.74%
1000	0.5121	83.94%	1.6681	52.24%

Πίνακας 18: Δείγματα Εποχών.

Epoch 1000/1000
 Training Loss: 0.5121, Training Accuracy: 83.94%
 Testing Loss: 1.6681, Testing Accuracy: 52.24%
 Total time until epoch 1000: 1955.28 seconds

Best Results:
 Epoch: 620
 Training Loss: 0.8128, Training Accuracy: 72.62%
 Testing Loss: 1.3793, Testing Accuracy: 53.95%
 Accuracy: 52.24%

Σχήμα 30: 2 hidden layers with 256 and 128 neurons and learning rate 0.001

3.1.3 3 hidden layers

- 256, 128, 64 νευρώνες - $\eta = 0.1$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **65.71 δευτερόλεπτα** στην εποχή 32/500 και ισούται με **testing accuracy=51.34%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.0291	26.00%	1.9273	30.44%
20	1.2778	54.41%	1.5019	46.66%
32	1.1288	59.71%	1.4250	51.34%
100	0.6134	77.90%	2.4265	46.64%
250	0.2303	92.01%	3.9071	48.12%
500	0.2775	92.04%	5.0715	46.89%

Πίνακας 19: Δείγματα Εποχών.

Epoch 500/500

Training Loss: 0.2775, Training Accuracy: 92.04%

Testing Loss: 5.0715, Testing Accuracy: 46.89%

Total time until epoch 500: 1052.16 seconds

Best Results:

Epoch: 32

Training Loss: 1.1288, Training Accuracy: 59.71%

Testing Loss: 1.4250, Testing Accuracy: 51.34%

Accuracy: 46.89%

Σχήμα 31: 3 hidden layers with 256, 128 and 64 neurons and learning rate 0.1

- 256, 128, 64 νευρώνες - $\eta = 0.01$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **194.02 δευτερόλεπτα** στην εποχή 102/500 και ισούται με **testing accuracy=53.69%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	1.9920	27.85%	1.8695	33.22%
60	1.0214	64.14%	1.3932	51.50%
102	0.7606	73.47%	1.4712	53.69%
250	0.1709	94.63%	2.7237	52.12%
300	0.0587	98.76%	3.1875	51.81%
500	0.0035	100.00%	4.1722	51.84%

Πίνακας 20: Δείγματα Εποχών.

Epoch 500/500
 Training Loss: 0.0035, Training Accuracy: 100.00%
 Testing Loss: 4.1722, Testing Accuracy: 51.84%
 Total time until epoch 500: 975.27 seconds

Best Results:
 Epoch: 102
 Training Loss: 0.7606, Training Accuracy: 73.47%
 Testing Loss: 1.4712, Testing Accuracy: 53.69%
 Accuracy: 51.84%

Σχήμα 32: 3 hidden layers with 256, 128 and 64 neurons and learning rate 0.01

- 256, 128, 64 νευρώνες - $\eta = 0.001$

Το καλύτερο **testing accuracy** σημειώνεται μετά από **736.74 δευτερόλεπτα** στην εποχή 376/1000 και ισούται με **testing accuracy=53.34%**

epoch	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
1	2.2168	19.76%	2.1376	24.83%
100	1.3746	51.73%	1.4594	48.36%
200	1.1763	58.55%	1.3711	51.77%
376	0.9173	67.74%	1.3818	53.34%
600	0.6558	77.42%	1.6462	51.15%
1000	0.3168	90.21%	2.2867	50.52%

Πίνακας 21: Δείγματα Εποχών.

Epoch 1000/1000
 Training Loss: 0.3168, Training Accuracy: 90.21%
 Testing Loss: 2.2867, Testing Accuracy: 50.52%
 Total time until epoch 1000: 1962.62 seconds

Best Results:
 Epoch: 376
 Training Loss: 0.9173, Training Accuracy: 67.74%
 Testing Loss: 1.3818, Testing Accuracy: 53.34%
 Accuracy: 50.52%

Σχήμα 33: 3 hidden layers with 256, 128 and 64 neurons and learning rate 0.001

3.1.4 ΤΕΛΙΚΕΣ ΠΑΡΑΤΗΡΗΣΕΙΣ ΔΟΚΙΜΩΝ ΓΙΑ ΣΤΑΘΕΡΟ LEARNING RATE

Αρχικά παρατηρούμε ότι το μεγαλύτερο testing accuracy που παρατηρήθηκε μετά από πειραματισμούς με τον αριθμό των hidden layers, με τον πειραματισμό του αριθμού των νευρώνων στα hidden layers, αλλά και με τον πειραματισμό με το learning rate, είναι **55.23%** για **1 hidden layer** με **512 νευρώνες** και **learning rate 0.001**.

Επίσης, όπως προαναφέρθηκε, παρατηρούμε σε κάθε δοκιμή για όλες τις περιπτώσεις πειραματισμού, μετά την επίτευξη του μέγιστου testing accuracy, ξεκινάει το φαινόμενο το overfitting, του οποίου ο ορισμός έχει δοθεί πιο πάνω.

Ωστόσο, ενδιαφέρον προκαλεί η εξής παρατήρηση: για μεγαλύτερο learning rate, το testing accuracy σημειώνει σημαντική μείωση μετά το overfitting. Αυτό μπορεί να ερμηνευτεί ως εξής: Το μοντέλο μαθαίνει γρήγορα τα δεδομένα εκπαίδευσης, αλλά μετά το overfitting, οι αλλαγές στα βάρη γίνονται υπερβολικά απότομες, οδηγώντας σε σημαντική πτώση του testing accuracy.

ΠΡΟΒΛΗΜΑ ΣΤΑΘΕΡΟΥ Learning Rate

Παρατηρούμε ότι για σταθερό learning rate και όσο πιο μεγάλο είναι αυτό, δημιουργούνται ταλαντώσεις στα αποτελέσματα του testing loss και του testing accuracy. Όταν έχουμε μεγάλο learning rate, τα βήματα που κάνει το μοντέλο κατά την διάρκεια της βελτιστοποίησης, δηλαδή οι αλλαγές που δημιουργούνται στα βάρη, είναι μεγάλα, με αποτέλεσμα το μοντέλο συχνά να υπερπηδά το ελάχιστο της συνάρτησης κόστους και καταλήγει να ταλαντώνεται γύρω από αυτό, αντί να συγκλίνει ομαλά. Όσο μειώνουμε το learning rate, βλέπουμε ότι οι ταλαντώσεις αυτές μειώνονται, αλλά αυτό γίνεται σε βάρος του χρόνου.

Θέλοντας να ξεπεραστεί το πρόβλημα των ταλαντώσεων, εφαρμόστηκε η εξής αλλαγή στο μοντέλο:

Πλέον, το **learning rate** με την πάροδο των εποχών μειώνεται με διάφορες τεχνικές. Στις παρακάτω παραγράφους παρουσιάζονται τα αποτελέσματα των δοκιμών για μεταβλητό learning rate με διάφορες τεχνικές μείωσης.

3.1.5 Τεχνική Step Decay

Με την τεχνική step decay, το learning rate μειώνεται σταδιακά κατά την διάρκεια της εκπαίδευσης σε συγκεκριμένα χρονικά διαστήματα (ανά αριθμό εποχών), έτσι ώστε το μοντέλο να κάνει πιο μικρά και ακριβή βήματα όσο πλησιάζει σε σύγκλιση.

Στον κώδικα, αυτό υλοποιείται ως εξής: Εισάγουμε 2 νέες μεταβλητές, το step_size και το decay_rate τα οποία καθορίζουν τον αριθμό των εποχών που μεσολαβούν ανάμεσα σε 2 μείωσεις του learning rate καθώς και τον συντελεστή της μείωσης αντίστοιχα και εισάγουμε τις εξής γραμμές κώδικα στην συνάρτηση train:

```
for epoch in range(epochs):
    # Μείωση Learning Rate κάθε step_size εποχές
    if epoch % step_size == 0 and epoch != 0:
        self.learning_rate *= decay_rate
        print(f"Epoch {epoch}: Reduced learning rate to {self.learning_rate:.5f}")

    train_batches = create_batches(X_train, y_train, batch_size)
```

Σχήμα 34: Step Decay

Δοκιμάστηκε αυτή η αλλαγή, όπως και οι υπόλοιπες που θα ακολουθήσουν, στο μοντέλο που χρησιμοποιεί 2 hidden layers με 256 και 128 νευρώνες αντίστοιχα, και θέσαμε σε κάθε περίπτωση αρχικό learning rate 0.01.

Συγκεκριμένα:

- **Step_size=10, Decay_rate=0.92**

Παρατηρούμε πράγματι ότι οι ταλαντώσεις ελαττώνονται σημαντικά χωρίς όμως να σταματούν να υπάρχουν. Σε αυτή τη περίπτωση το μέγιστο testing accuracy σημειώνεται στην εποχή 213 για 417.90 δευτερόλεπτα και είναι 54.37%. Για χάρην σύγκρισης, για το ίδιο μοντέλο με σταθερά learning rates είχαμε:

- Για $\eta=0.1$ testing accuracy=51.74% στα 98.73 δευτερόλεπτα
- Για $\eta=0.01$ testing accuracy=53.86% στα 162.48 δευτερόλεπτα
- Για $\eta=0.001$ testing accuracy=53.95% στα 1210.46 δευτερόλεπτα

Οπότε, καταφέραμε να αυξήσουμε το testing accuracy χωρίς να αυξάνουμε τον χρόνο που απαιτείται για να επιτευχθεί, και ελαττώσαμε τις ταλαντώσεις.

```
Epoch 700/700
Training Loss: 0.4640, Training Accuracy: 86.71%
Testing Loss: 1.5863, Testing Accuracy: 53.50%
Total time until epoch 700: 1381.38 seconds
-----
Best Results:
Epoch: 213
Training Loss: 0.6505, Training Accuracy: 78.86%
Testing Loss: 1.4411, Testing Accuracy: 54.37%
Accuracy: 53.50%
```

Σχήμα 35: Step Size=10, decay rate=0.92

- **Step_size=5, Decay_rate=0.95**

Παρότι με την προηγούμενη δοκιμή μειώθηκαν οι ταλαντώσεις και η απόδοση αυξήθηκε, δεν ήταν αρκετά ικανοποιητικό το αποτέλεσμα. Σε αυτή τη δοκιμή, το learning rate μειώνεται πιο συχνά, συγκεκριμένα κάθε 5 εποχές με decay rate 0.96

Πράγματι, σε αυτή τη περίπτωση το πρόβλημα των ταλαντώσεων είναι αρκετά μικρότερο (ωστόσο εξακολουθεί να υπάρχει). Αυτό οφείλεται στο γεγονός ότι το learning rate μειώνεται πιο γρήγορα σε μικρότερες τιμές, γεγονός που οδηγεί σε πιο ακριβείς ενημερώσεις βαρών.

Το testing accuracy σε αυτή τη περίπτωση παρατηρείται την εποχή 175 για 337.79 δευτερόλεπτα και είναι 54.87%. Όποτε παρατηρούμε ότι το learning rate αυξάνεται ακόμη περισσότερο για μικρότερο step size, ενώ ο χρόνος επίτευξης τους μειώνεται.

```
Epoch 500/500
Training Loss: 0.6092, Training Accuracy: 80.67%
Testing Loss: 1.4314, Testing Accuracy: 53.99%
Total time until epoch 500: 985.56 seconds
-----
Best Results:
Epoch: 243
Training Loss: 0.6876, Training Accuracy: 77.39%
Testing Loss: 1.3922, Testing Accuracy: 54.74%
Accuracy: 53.99%
```

Σχήμα 36: Step Size=5, decay rate=0.95

3.1.6 Τεχνική Exponential Decay

Σε αυτή την τεχνική μείωσης του learning rate, αυτό μειώνεται συνεχώς με βάση μια εκθετική συνάρτηση, και όχι διακριτά όπως στο step decay.

Η εξίσωση για το **Exponential Decay** είναι:

$$\eta_t = \eta_0 \cdot e^{-\lambda \cdot t}$$

Όπου:

- η_t : Το learning rate στη εποχή t .

- η_0 : Το αρχικό learning rate.
- λ : Ο ρυθμός μείωσης (decay rate). Ρυθμίζει πόσο γρήγορα μειώνεται το learning rate.
- t : Ο αριθμός των εποχών εκπαίδευσης.

Το exponential decay, υλοποιείται εντός της συνάρτησης train ως εξής:

```
for epoch in range(epochs):
    # Μείωση Learning Rate με Exponential Decay
    self.learning_rate = self.initial_learning_rate * np.exp(-decay_rate * epoch)

    train_batches = create_batches(X_train, y_train, batch_size)
```

Σχήμα 37: Exponential Decay

- **Decay_rate=0.005**

Αρχικά, αυτή η τεχνική δοκιμάστηκε για decay_rate=0.005. Παρατηρείται ότι η τεχνική αυτή αντιμετωπίζει αποτελεσματικότερα το φαινόμενο των ταλαντώσεων λόγω του ότι το learning rate μειώνεται σταδιακά και όχι απότομα, αποφεύγοντας μεγάλες αλλαγές που μπορεί να αποσταθεροποιήσουν την εκπαίδευση.

Το testing accuracy σε αυτή τη περίπτωση φτάνει την τιμή 54.42% την εποχή 135 και για χρόνο 265.78 δευτερόλεπτα, σχεδόν ίση με τις τιμές των testing accuracy που είδαμε στην τεχνική step decay, αλλά αρκετά ταχύτερο.

```
Epoch 500/500
Training Loss: 0.2150, Training Accuracy: 95.75%
Testing Loss: 2.0414, Testing Accuracy: 52.05%
Current Learning Rate: 0.00082
Total time until epoch 500: 1004.66 seconds

-----

Best Results:
Epoch: 135
Training Loss: 0.7673, Training Accuracy: 73.93%
Testing Loss: 1.3863, Testing Accuracy: 54.42%
Accuracy: 52.05%
```

Σχήμα 38: Decay rate 0.005

- **Decay_rate=0.01**

Έπειτα, δοκίμασα να αυξήσω το decay rate σε 0.01. Αυξάνοντας το decay rate, παρατηρείται μια ελαφρά βελτίωση αναφορικά με το φαινόμενο των ταλαντώσεων.

Το testing accuracy σε αυτή τη περίπτωση σημείωσε μια απειροελάχιστη αύξηση, 54.36%, αλλά ο χρόνος επίτευξης του μειώνεται ακόμα περισσότερο, σε 201.31 δευτερόλεπτα.

```

Epoch 200/200
Training Loss: 0.7266, Training Accuracy: 76.17%
Testing Loss: 1.3955, Testing Accuracy: 53.30%
Current Learning Rate: 0.00137
Total time until epoch 200: 376.93 seconds
=====

```

```

Best Results:
Epoch: 107
Training Loss: 0.9326, Training Accuracy: 67.96%
Testing Loss: 1.3316, Testing Accuracy: 54.36%
Accuracy: 53.30%

```

Σχήμα 39: Decay rate 0.01

ΣΥΜΠΕΡΑΣΜΑ Έπειτα από όλες τις δοκιμές που προηγήθηκαν, είναι φανερό πως η αποτελεσματικότερη τεχνική είναι αυτή του exponential decay, καθώς το φαινόμενο των ταλαντώσεων δείχνει να βελτιώνεται αρκετά σε σχέση με την περίπτωση του σταθερού learning rate, ενώ το testing accuracy σημειώνει βελτίωση τόσο στο ποσοστό, όσο και στον χρόνο που επιτυγχάνεται.

4 Σύγκριση Νευρωνικού με κατηγοριοποίηση Nearest Neighbor και Nearest Class Centroid

Οι κατηγοριοποιητές πλησιέστερου γείτονα (Nearest Neighbor) και πλησιέστερου κέντρου κλάσης (Nearest Class Centroid) είναι μέθοδοι κατηγοριοποίησης που βασίζονται στη γεωμετρική εγγύτητα στον χώρο χαρακτηριστικών. Ο κατηγοριοποιητής Nearest Neighbor λειτουργεί με την υπόθεση ότι η κλάση ενός δείγματος καθορίζεται από την κλάση του πλησιέστερου δείγματος εκπαίδευσης, μετρώντας ευκλείδειες αποστάσεις. Αντίθετα, ο κατηγοριοποιητής Nearest Class Centroid βασίζεται στον υπολογισμό του κέντρου βάρους (centroid) κάθε κλάσης στα δεδομένα εκπαίδευσης, αντιστοιχίζοντας ένα νέο δείγμα στην κλάση του κοντινότερου centroid.

Κατηγοριοποιητής	Accuracy	Time
1-Nearest Neighbor	35.39%	2412 sec
3-Nearest Neighbor	33.03%	1981 sec
Nearest Centroid	27.74%	0.6 sec

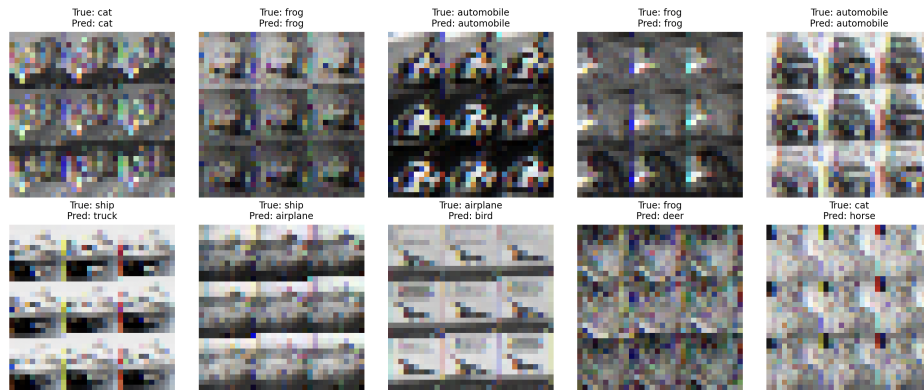
Πίνακας 22: 1-NN, 3-NN, Nearest Centroid

Σχετικά με τον κατηγοριοποιητή **Nearest Neighbor**, παρατηρούμε ότι τόσο χρονικά, όσο και από άποψη accuracy, είναι ασύμφορο να χρησιμοποιηθεί έναντι του νευρωνικού δικτύου που παρουσιάστηκε σε αυτή την εργασία, καθώς το νευρωνικό δίκτυο μπορεί και "αγγίζει" accuracy έως και 20% περισσότερο, για αρκετά πιο σύντομο χρονικό διάστημα.

Ωστόσο, ο κατηγοριοποιητής Nearest Neighbor αποτελεί ένα είδος κατηγοριοποιητή πολύ απλού στην κατανόηση και στην υλοποίηση, και αυτό είναι το θετικό του έναντι του νευρωνικού δικτύου, το οποίο παρότι έχει πολύ καλύτερες επιδόσεις, είναι είναι δύσκολο στην κατανόηση και περίπλοκο στην υλοποίηση.

Αναφορικά με τον κατηγοριοποιητή **Nearest Centroid**, αποτελεί τον χειρότερο κατηγοριοποιητή από άποψη απόδοσης, ωστόσο ο χρόνος που μπορεί και επιτυγχάνει αυτή την ακρίβεια θα μπορούσε να αποτελέσει αφορμή για την επιλογή του σε περιπτώσεις που το accuracy δεν αποτελεί προτεραιότητα.

5 Παράδειγμα Ορθής και Εσφαλμένης Κατηγοριοποίησης.



Σχήμα 40: Παραδείγματα Κατηγοριοποίησης

Στην εικόνα φαίνεται ένα παράδειγμα με 5 ορθές και 5 εσφαλμένες κατηγοριοποιήσεις. Οι εικόνες εμφανίζονται θολές επειδή τα δεδομένα CIFAR-10 περιέχουν μικρές εικόνες μεγέθους 32x32 pixels, και όταν οι εικόνες αναπαρίστανται σε μεγαλύτερες διαστάσεις, τα pixels “τεντώνονται”, κάνοντάς τις να φαίνονται θολές.